

9 out of 10 GenAI engineers can't answer these questions.

The questions that actually get asked in AI engineering interviews and why most candidates freeze.

Swipe to find out →

Where AI Interview Prep Goes Wrong



Studying theory without understanding trade-offs

Interviewers ask why, not what. "Attention is $O(n^2)$ " is not an answer.



Cannot connect math to system design decisions

Probability theory, embeddings, and loss functions show up in design discussions



Weak on fine-tuning, RAG, and deployment trade-offs

Every senior AI role asks you to choose and justify the right approach



No production engineering depth

FastAPI, async patterns, and scaling are tested at every level above junior



This kit covers every layer tested, from math foundations to deployment architecture.

300+ Questions Across 8 Topic Areas

Structured to match how companies actually run AI engineering interviews.

✓ AI Fundamentals & Math

⚡ Transformers In-Depth



LLM Architecture & Behavior



Prompt Engineering



Fine-Tuning & Adaptation



GenAI System Design



Memory, Vector DBs & Caching



Backend & Deployment



Check the caption for the AI Interview Kit.

Q01

When would you choose fine-tuning over prompt engineering?

💡 KEY REASONING

- ✓ Try prompt engineering first. Fine-tuning has real costs and is only justified when prompting cannot deliver consistently.
- ✓ **Format at scale:** If few-shot examples cost 1,500 tokens per request, fine-tuning internalizes the format and eliminates them at 1M requests per day.
- ✓ **Conflicting behavior:** When you need responses that fight the model's default behavior, fine-tuning changes the base, prompting just fights it.
- ✓ **Domain knowledge:** Proprietary internal terminology not represented in base training. Prompting cannot inject knowledge, only behavior.

→ Start with prompts. Fine-tune only when prompting genuinely cannot meet the requirement.

🔖 Check caption for the AI Interview Kit

Q02

Why does the transformer rely entirely on attention instead of recurrence?

💡 CORE INSIGHT

- ✓ **RNNs process sequentially.** Long-range dependencies require propagating information through many steps, losing signal over distance.
- ✓ **Attention gives every token direct access to every other token.** No path length penalty regardless of sequence position distance.
- ✓ **Parallelism:** Unlike RNNs, all attention computations run simultaneously on GPU. Training transformers is orders of magnitude faster.
- ✓ **Trade-off:** Attention is $O(n^2)$ in sequence length. Positional encodings must be added explicitly since there is no built-in order.



Attention solves the fundamental sequential bottleneck of RNNs by enabling full parallel access.

🔖 Check caption for the AI Interview Kit

Q03

How would you design a production GenAI system for thousands of concurrent users?

💡 ARCHITECTURE LAYERS

- ✓ **Entry layer:** API gateway handles auth, rate limiting per user, and routing. Every request gets a unique trace ID here.
- ✓ **LLM proxy:** Dedicated service in front of all model calls handles retries, fallbacks to backup providers, cost tracking, and response caching.
- ✓ **Async for long tasks:** Agent workflows and document ingestion go through a job queue. Never block the user-facing path.
- ✓ **Data layer:** Vector store for retrieval, Redis for session cache and semantic response cache, relational DB for user data.



Never let the LLM sit in the critical path without a proxy, cache, and fallback layer.

🔖 Check caption for the AI Interview Kit

Q04

How do temperature, top-k, and top-p interact during token generation?

💡 SAMPLING PIPELINE

- ✓ **Execution order:** Logits produced by model, scaled by temperature, filtered by top-k or top-p, softmax applied, then sampled.
- ✓ **Temperature:** Scales logit distribution. High temperature flattens it (more creative). Low temperature sharpens it (more deterministic).
- ✓ **top-k:** Restricts sampling to the k highest probability tokens. top-p instead takes the minimum set of tokens whose cumulative probability exceeds p.
- ✓ **Temperature 0 does not guarantee determinism** across all inference systems due to GPU floating-point parallelism and batching effects.

→ Understand the order and interaction. Interviewers probe whether you know the implementation details.

🔖 Check caption for the AI Interview Kit

Q05

When would you use zero-shot vs few-shot prompting? What changes between them?

💡 DECISION FRAMEWORK

- ✓ **Zero-shot:** No examples. Works well for tasks the model has seen extensively in training: translation, summarization, simple classification. Start here always.
- ✓ **One-shot:** One example clarifies format and tone without heavy token cost. Best when your output structure differs from the model's default.
- ✓ **Few-shot:** Two to five examples. The model infers the pattern. Best for complex classification, nuanced tone matching, or domain-specific extraction.
- ✓ **Risk:** If examples are not representative, the model overfits to the examples rather than generalizing to the underlying task.

[🔖 Check caption for the AI Interview Kit](#)

Q06

How would you design short-term and long-term memory for an AI agent?

💡 MEMORY ARCHITECTURE

- ✓ **Short-term memory lives in the context window.** It contains the task description, the plan, tool call results, and intermediate conclusions. Token-limited, so must be actively managed.
- ✓ **Manage context size:** Track token usage. Once a step is complete, replace verbose output with a compressed summary to avoid running out mid-task.
- ✓ **Long-term memory lives externally:** Vector DB for semantic retrieval, relational DB for structured facts. Retrieved at task start, written at task completion.
- ✓ **Graduation:** Short-term insights that prove durably important get summarized and promoted to long-term memory for future tasks.

[🔖 Check caption for the AI Interview Kit](#)

Q07

How do you handle high-throughput LLM requests in FastAPI without blocking the event loop?

💡 ARCHITECTURE SPLIT

- ✓ **External APIs (OpenAI, Anthropic):** These are I/O-bound. Use `async def` with an async HTTP client like `httpx`. Never use the synchronous requests library inside async endpoints.
- ✓ **Self-hosted models:** Local inference is GPU-bound, not I/O-bound. Running it inside the FastAPI event loop blocks the server. Use a separate inference service (vLLM, TGI) called via HTTP.
- ✓ **Concurrency control:** Use semaphores or rate limiters to prevent upstream overload when many requests arrive simultaneously.
- ✓ **Streaming:** Return tokens as they are generated using `StreamingResponse`. Reduces perceived latency for users without changing total generation time.

[🔖 Check caption for the AI Interview Kit](#)

Q08

What is the difference between encoder-only, decoder-only, and encoder-decoder models?

💡 ARCHITECTURE ROLES

- ✓ **Encoder-only (BERT):** Sees the full sequence bidirectionally. Best for classification, named entity recognition, and embedding generation. Cannot generate text autoregressively.
- ✓ **Decoder-only (GPT, LLaMA):** Generates tokens left-to-right with causal masking. Best for text generation, chat, and instruction following. The dominant architecture for LLMs today.
- ✓ **Encoder-decoder (T5, BART):** Encoder reads the input, decoder generates output. Best for translation, summarization, and structured output tasks.
- ✓ **Why decoder-only won:** Scaling laws favor decoder-only architectures. With sufficient scale and RLHF, they match or exceed encoder-decoder on most tasks.

[🔖 Check caption for the AI Interview Kit](#)

Q09

When would you choose RAG over fine-tuning to add domain knowledge to an LLM?

💡 DECISION FRAMEWORK

- ✓ **RAG for dynamic knowledge:** If the knowledge changes frequently (news, product catalog, legal updates), RAG lets you update the knowledge base without retraining.
- ✓ **RAG for verifiability:** RAG lets you show users exactly which source documents were used. Fine-tuning bakes knowledge in without a clear audit trail.
- ✓ **Fine-tuning for style and behavior:** If you need consistent output format, tone, or a specialized reasoning pattern, fine-tuning is more effective than retrieval.
- ✓ **Use both:** The strongest production systems combine fine-tuning for behavior with RAG for dynamic knowledge. They are complementary, not competing.

[🔖 Check caption for the AI Interview Kit](#)

Q10

Why does Bayes' theorem matter in machine learning, and where does it show up concretely?

💡 PRACTICAL APPLICATIONS

- ✓ **Class imbalance:** Fraud detection with 0.1% fraud rate. A model ignoring base rates classifies borderline transactions as fraud even when the prior probability of fraud is very low.
- ✓ **In training:** The model learns $P(\text{output} \text{ given input})$. Cross-entropy loss is a direct log-probability measure derived from Bayesian reasoning.
- ✓ **Naive Bayes classifiers:** Still competitive for text classification. The class with the highest posterior probability given the features wins.
- ✓ **Bayesian deep learning:** Models output distributions over predictions, not just point estimates. Critical for medical and autonomous systems where uncertainty quantification matters.

[🔖 Check caption for the AI Interview Kit](#)

AI & GENAI ENGINEER KIT

300+ questions. Every layer actually tested.

From probability theory and transformers through to production system design and FastAPI deployment. Structured to match how companies run AI engineering interviews.

↓ **Check the Caption**

To get the complete AI & GenAI Engineer
Interview Kit